

STA623 - Bayesian Data Analysis

Practical 4: MCMC with NIMBLE and convergence diagnostics (with solutions)

Marc Henrion

2026-10-01

Goals. Specify and run models in NIMBLE; validate MCMC output against known answers; assess convergence with trace plots, $\widehat{R}$, bulk/tail ESS and MCSE; diagnose and fix poor mixing. Accompanies Lecture 4.

The helper functions `mcmc_to_long()` and `trace_plot()` from the lecture notes are defined in the (hidden) setup chunk of this document.

Exercise 1: validating NIMBLE against an exact answer

Malaria episodes for 18 children:

```
y <- c(1, 3, 2, 3, 0, 2, 6, 4, 4, 1, 1, 3, 2, 3, 1, 1, 3, 0)
```

- (a) Fit $Y_i \sim \text{Pois}(\lambda)$, $\lambda \sim \text{Gamma}(5, 2)$ in NIMBLE with 4 chains. Which sampler does NIMBLE choose for λ , and why?
- (b) Compare the posterior mean, 95% equal-tailed interval and 95% HPD interval from the MCMC draws with the exact $\text{Gamma}(45, 20)$ values. Are differences within Monte Carlo error?
- (c) Now put the prior on $\theta = \log \lambda$: $\theta \sim \mathcal{N}(0, 1.5^2)$. Which sampler is used now? Compare the posterior of λ with (b).

Solution

(a)

```
gp_code <- nimbleCode({
  for (i in 1:N) { y[i] ~ dpois(lambda) }
  lambda ~ dgamma(5, 2)
})
gp_model <- nimbleModel(gp_code, constants = list(N = length(y)), data = list(y =
  ↪ y),
                                inits = list(lambda = 1))
gp_conf <- configureMCMC(gp_model)
```

```
gp_conf$printSamplers()
```

```
[1] conjugate_dgamma_dpois_identity sampler: lambda
```

NIMBLE detects the Gamma-Poisson conjugacy and uses an exact conjugate (Gibbs) sampler, which draws directly from the $\text{Gamma}(45, 20)$ full conditional.

```
gp_c <- compileNimble(gp_model)
gp_mcmc <- compileNimble(buildMCMC(gp_conf), project = gp_model)
gp_fit <- runMCMC(gp_mcmc, niter = 6000, nburnin = 1000, nchains = 4,
  inits = function() list(lambda = runif(1, 0.5, 5)),
  setSeed = TRUE, progressBar = FALSE, samplesAsCodaMCMC = TRUE)
```

(b)

```
lam <- as.matrix(gp_fit)[, "lambda"]
rbind(MCMC = c(mean = mean(lam), quantile(lam, c(0.025, 0.975)), hdi(lam)),
  exact = c(45 / 20, qgamma(c(0.025, 0.975), 45, 20), hdi(qgamma, 0.95, shape
    ↪ = 45, rate = 20))) |>
  kable(digits = 4)
```

	mean	2.5%	97.5%	lower	upper
MCMC	2.2538	1.6469	2.9595	1.6177	2.9219
exact	2.2500	1.6412	2.9534	1.6113	2.9171

```
summarise_draws(as_draws(gp_fit), "mean", "mcse_mean", ~mcse_quantile(.x, probs =
  ↪ c(0.025, 0.975)), "ess_bulk")
```

```
# A tibble: 1 x 6
  variable mean mcse_mean mcse_q2.5 mcse_q97.5 ess_bulk
  <chr>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
1 lambda    2.25    0.00241  0.00473  0.00796  19383.
```

The differences are of the order of the MCSEs. Because the conjugate sampler produces independent draws, ESS is close to the total number of draws (20,000).

(c)

```
ln_code <- nimbleCode({
  for (i in 1:N) { y[i] ~ dpois(lambda) }
  lambda <- exp(theta)
  theta ~ dnorm(0, sd = 1.5)
})
ln_model <- nimbleModel(ln_code, constants = list(N = length(y)), data = list(y =
  ↪ y),
                                inits = list(theta = 0))
ln_conf <- configureMCMC(ln_model, monitors = c("theta", "lambda"))
```

```
ln_conf$printSamplers()
```

```
[1] RW sampler: theta
```

```
ln_c <- compileNimble(ln_model)
ln_mcmc <- compileNimble(buildMCMC(ln_conf), project = ln_model)
ln_fit <- runMCMC(ln_mcmc, niter = 6000, nburnin = 1000, nchains = 4,
  inits = function() list(theta = rnorm(1)),
  setSeed = TRUE, progressBar = FALSE, samplesAsCodaMCMC = TRUE)
```

```
summarise_draws(as_draws(ln_fit), "mean", ~quantile(.x, c(0.025, 0.975)), "rhat",
  ↪ "ess_bulk")
```

```
# A tibble: 2 x 6
  variable mean `2.5%` `97.5%` rhat ess_bulk
  <chr>     <dbl> <dbl>   <dbl> <dbl>   <dbl>
1 lambda   2.20    1.56    2.94  1.00    4341.
2 theta    0.777   0.447    1.08  1.00    4341.
```

With no conjugacy, NIMBLE uses an adaptive random-walk Metropolis sampler. The ESS is now noticeably smaller than the number of draws (autocorrelation), but still ample. The posterior for λ is very close to (b): with 18 observations both priors are weak relative to the data.

Exercise 2: what bad diagnostics look like

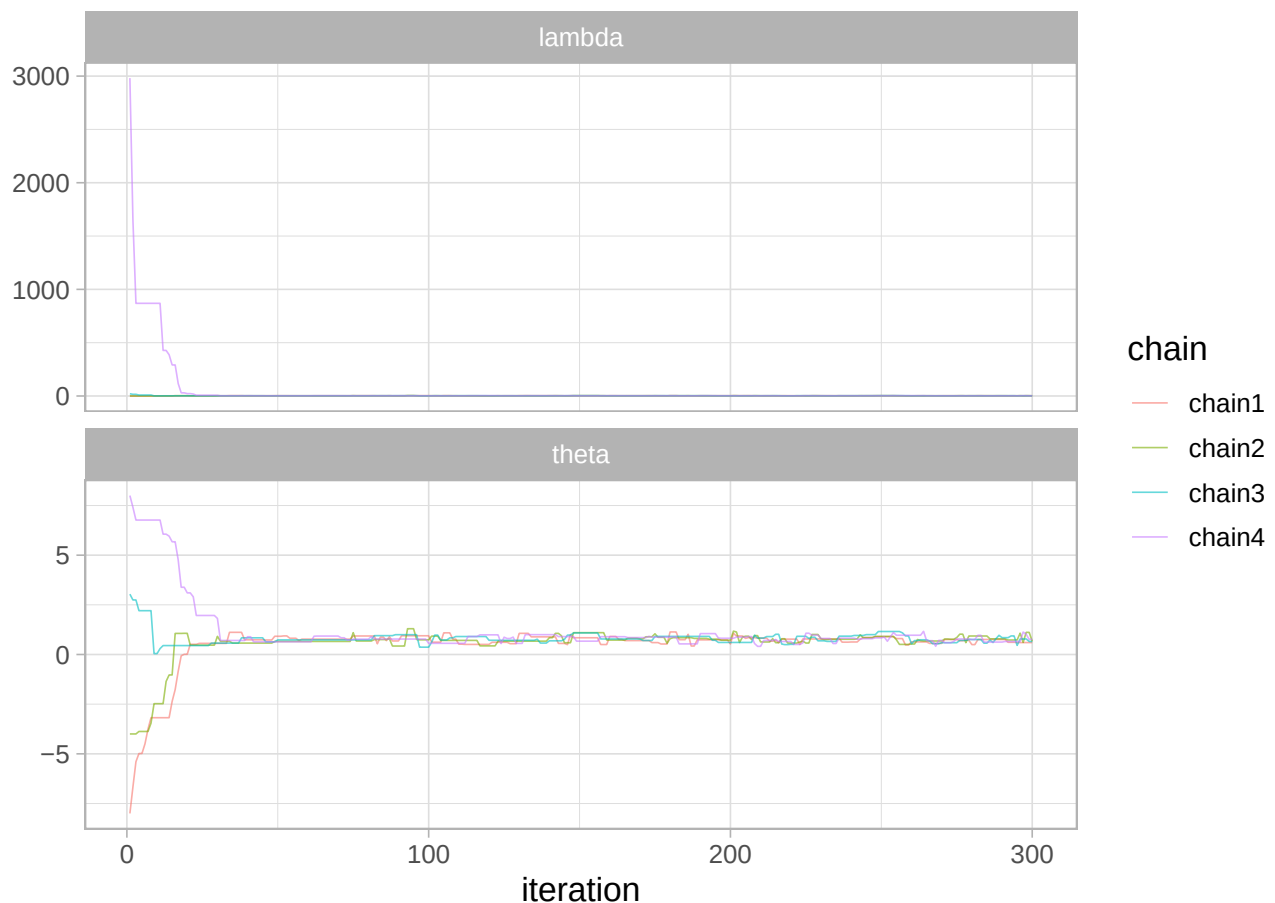
Using the model of Exercise 1(c):

- (a) Run 4 chains of only 300 iterations, **no burn-in**, starting from $\theta = -8, -4, 4, 8$. Plot the traces and compute $\widehat{R}$ and ESS. Would you trust these results?
- (b) What is the smallest burn-in you would use, judging from the trace plot? Rerun with 2,000 iterations and that burn-in. Are the diagnostics now acceptable?

Solution

(a)

```
bad_inits <- list(list(theta = -8), list(theta = -4), list(theta = 4), list(theta  
  ↪ = 8))  
bad_fit <- runMCMC(ln_mcmc, niter = 300, nburnin = 0, nchains = 4, inits =  
  ↪ bad_inits,  
                  setSeed = TRUE, progressBar = FALSE, samplesAsCodaMCMC = TRUE)  
  
trace_plot(bad_fit)
```



```
summarise_draws(as_draws(bad_fit), "mean", "rhat", "ess_bulk", "ess_tail")
```

```
# A tibble: 2 x 5
  variable  mean  rhat ess_bulk ess_tail
  <chr>    <dbl> <dbl>   <dbl>   <dbl>
1 lambda  14.3    1.06    51.3    50.1
2 theta   0.758   1.06    51.3    50.1
```

The chains start far apart and take a number of iterations (while the sampler is still adapting its step size) to reach the posterior. Including these iterations inflates $\hat{R}$ well above 1.01 and ruins ESS, especially tail-ESS. The posterior mean is badly biased. These results should not be used.

(b) The traces settle after roughly 100-200 iterations; a burn-in of 500 gives a margin of safety.

```
ok_fit <- runMCMC(ln_mcmc, niter = 2000, nburnin = 500, nchains = 4, inits =
  ↪ bad_inits,
               setSeed = TRUE, progressBar = FALSE, samplesAsCodaMCMC = TRUE)
```

```
summarise_draws(as_draws(ok_fit), "mean", "rhat", "ess_bulk", "ess_tail")
```

```
# A tibble: 2 x 5
  variable  mean  rhat ess_bulk ess_tail
  <chr>    <dbl> <dbl>   <dbl>   <dbl>
1 lambda   2.21   1.01   1221.   1183.
2 theta    0.780   1.01   1221.   1183.
```

$\hat{R}$ is now at or just below 1.01 and ESS exceeds 1,000: acceptable, though running a little longer would give a comfortable margin. Dispersed starting values are *useful*: if chains started in very different places end up agreeing, we have much stronger evidence of convergence than from a single chain.

Exercise 3: logistic regression that mixes badly

Generate data:

```
set.seed(2026)
N <- 100
age <- runif(N, 20, 60) # age in years; far from 0!
p <- plogis(-6 + 0.12 * age)
seropos <- rbinom(N, 1, p)
```

- (a) Fit $\text{logit } P(Y_i = 1) = \beta_0 + \beta_1 \text{age}_i$ with $\beta_0 \sim \mathcal{N}(0, 10^2)$, $\beta_1 \sim \mathcal{N}(0, 1)$ in NIMBLE: 4 chains, 3,000 iterations, burn-in 1,000. Compute $\widehat{R}$, bulk and tail ESS; look at trace plots, autocorrelation (`coda::autocorr.plot`) and a scatterplot of β_0 against β_1 .
- (b) Explain the problem. Fix it by centring age (and expressing it per 10 years). Refit and compare diagnostics.
- (c) Alternatively keep the original parameterisation but replace the default samplers with a joint `AF_slice` sampler on (β_0, β_1) . Compare.
- (d) From the well-mixing fit, report the posterior median and 95% equal-tailed and HPD intervals for the odds ratio per 10 years of age, and compare with `glm()`.

Solution

(a)

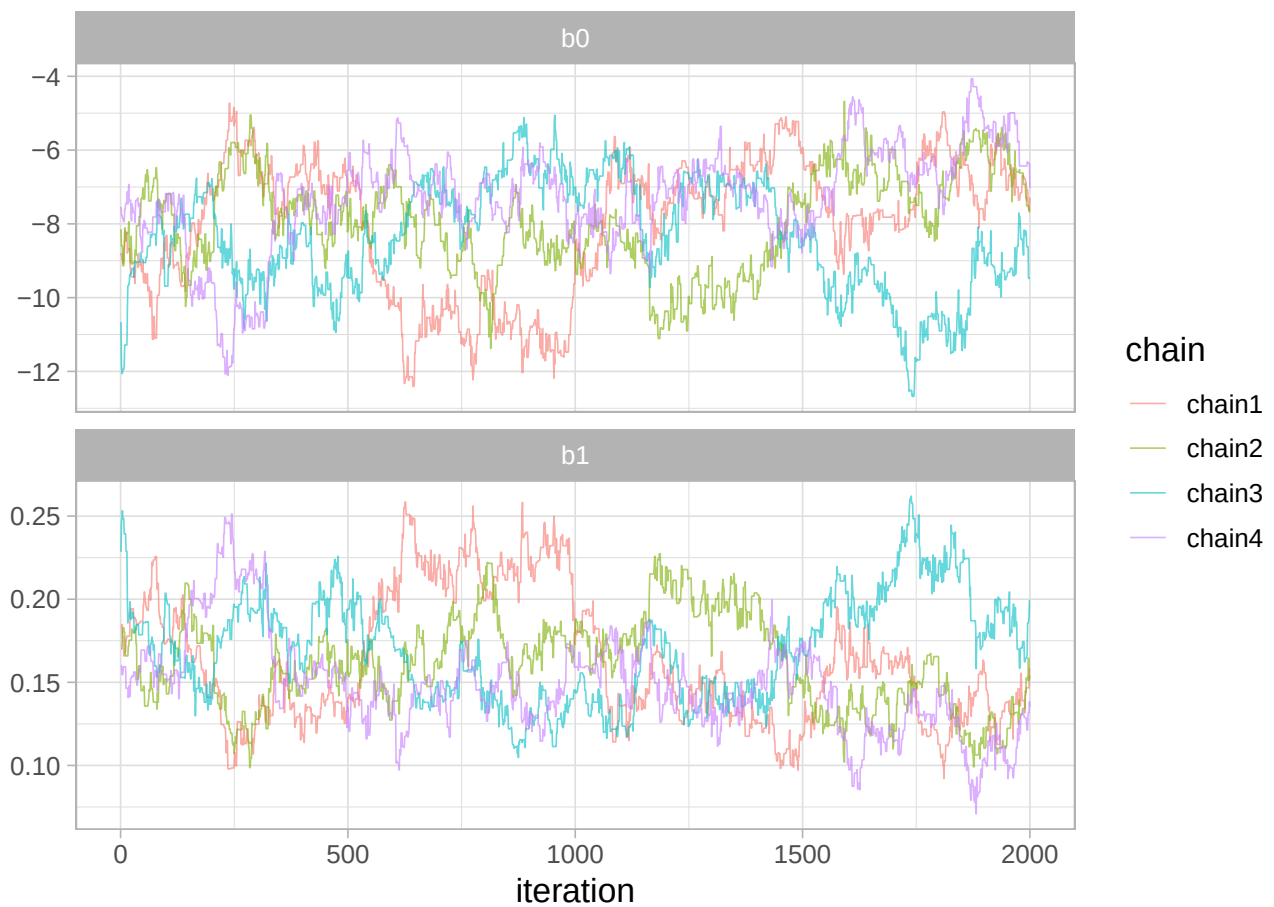
```
logit_code <- nimbleCode({
  for (i in 1:N) {
    y[i] ~ dbern(p[i])
    logit(p[i]) <- b0 + b1 * x[i]
  }
  b0 ~ dnorm(0, sd = 10)
  b1 ~ dnorm(0, sd = 1)
})
inits_fun <- function() list(b0 = rnorm(1, 0, 2), b1 = rnorm(1, 0, 0.1))

fit_raw <- nimbleMCMC(logit_code, constants = list(N = N, x = age), data = list(y
  ↪ = seropos),
  inits = inits_fun, monitors = c("b0", "b1"), nchains = 4,
  niter = 3000, nburnin = 1000, setSeed = TRUE, progressBar =
  ↪ FALSE,
  samplesAsCodaMCMC = TRUE)

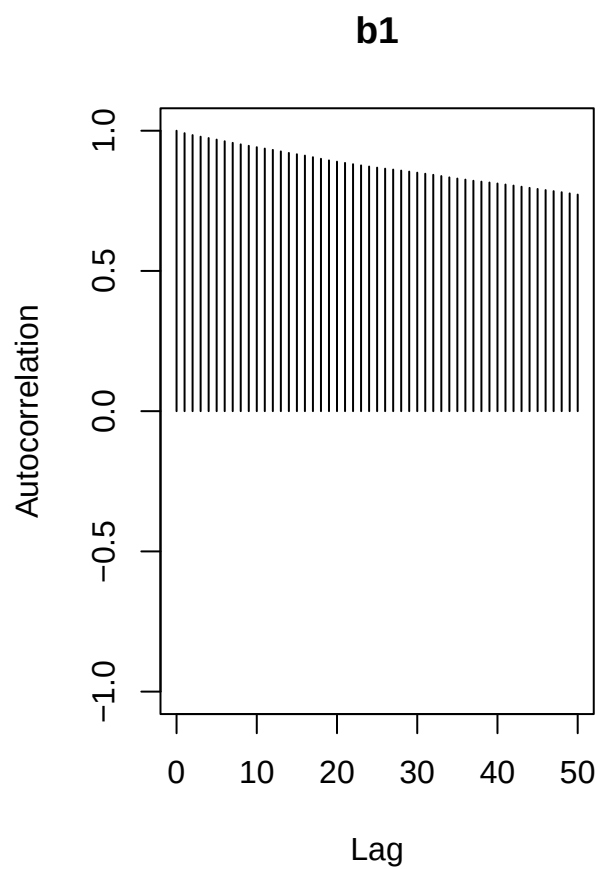
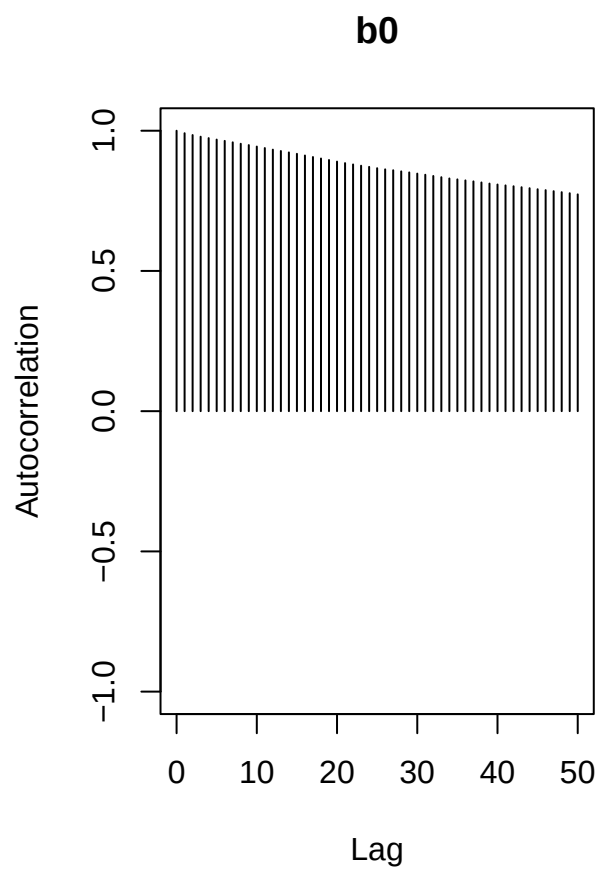
summarise_draws(as_draws(fit_raw), "mean", "sd", "rhat", "ess_bulk", "ess_tail")
```

```
# A tibble: 2 x 6
  variable    mean      sd  rhat ess_bulk ess_tail
  <chr>      <dbl>   <dbl> <dbl>   <dbl>   <dbl>
1 b0        -7.90    1.55   1.12    27.7    45.5
2 b1         0.160 0.0329  1.11    27.8    47.0
```

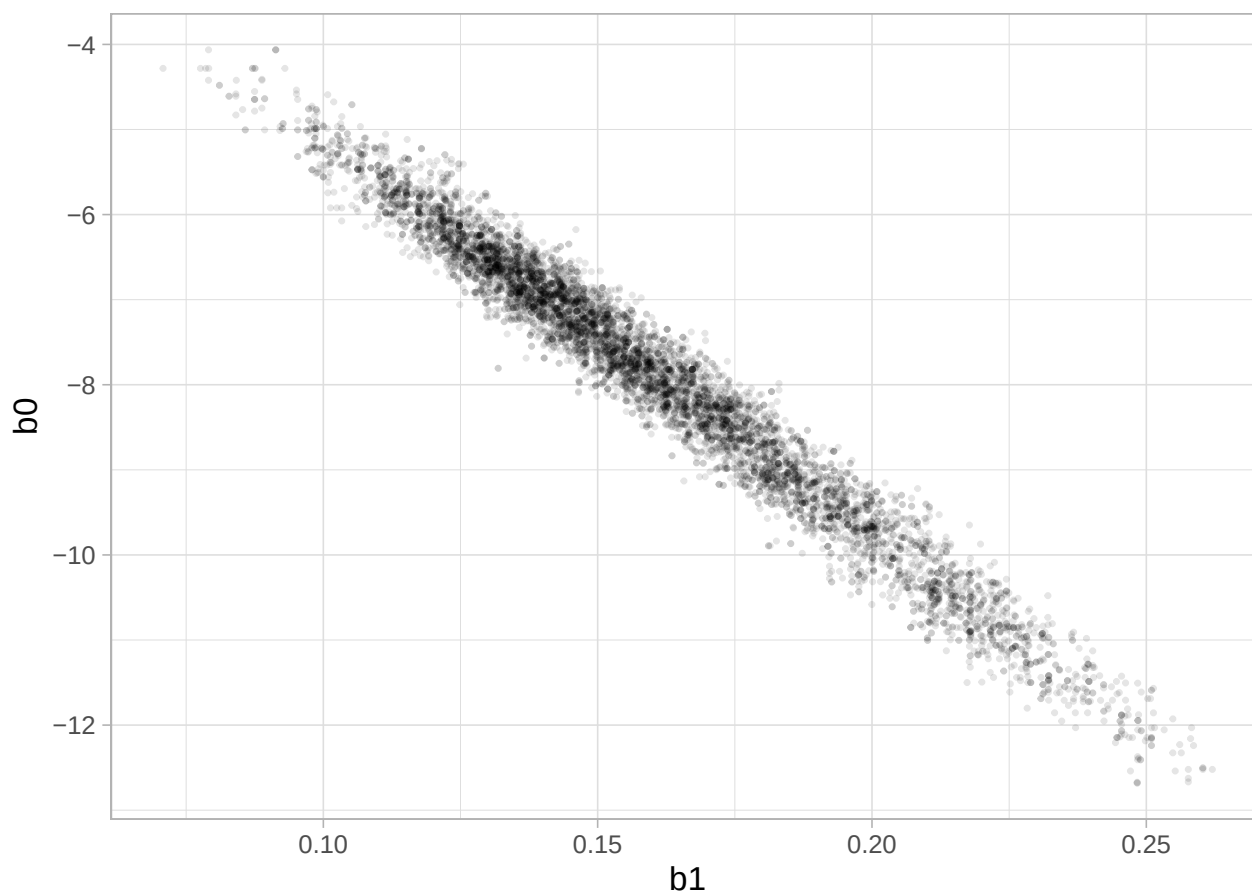
```
trace_plot(fit_raw)
```



```
autocorr.plot(fit_raw[[1]], lag.max = 50)
```



```
as_tibble(as.matrix(fit_raw)) |> ggplot(aes(b1, b0)) + geom_point(alpha = 0.1,  
  ↵ size = 0.5)
```



- (b) Age ranges from 20 to 60, so the intercept is the log-odds at age 0, far outside the data. It is almost perfectly negatively correlated with the slope, and one-at-a-time random-walk updates can only take tiny steps along this ridge: very high autocorrelation, low ESS, possibly $\hat{R} > 1.01$. Centring makes the intercept the log-odds at the mean age and removes most of the correlation.

```
age10_c <- (age - mean(age)) / 10
fit_c <- nimbleMCMC(logit_code, constants = list(N = N, x = age10_c), data =
  ↪ list(y = seropos),
      inits = function() list(b0 = rnorm(1), b1 = rnorm(1)),
      monitors = c("b0", "b1"), nchains = 4, niter = 3000, nburnin
        ↪ = 1000,
      setSeed = TRUE, progressBar = FALSE, samplesAsCodaMCMC = TRUE)
```

```
summarise_draws(as_draws(fit_c), "mean", "sd", "rhat", "ess_bulk", "ess_tail")
```

```
# A tibble: 2 x 6
  variable mean    sd  rhat ess_bulk ess_tail
  <chr>    <dbl> <dbl> <dbl>    <dbl>    <dbl>
1 b0      -1.49 0.341  1.01     904.     1030.
2 b1       1.44 0.296  1.00     935.     1161.
```

Note that the prior $\beta_1 \sim \mathcal{N}(0, 1)$ now refers to the log-OR per 10 years, a more natural scale on which to think about priors.

(c)

```
m <- nimbleModel(logit_code, constants = list(N = N, x = age), data = list(y =
  ↪ seropos),
                inits = inits_fun())
conf <- configureMCMC(m, monitors = c("b0", "b1"))
conf$removeSamplers(c("b0", "b1"))
conf$addSampler(target = c("b0", "b1"), type = "AF_slice")
cm <- compileNimble(m)
cmcmc <- compileNimble(buildMCMC(conf), project = m)
fit_af <- runMCMC(cmcmc, niter = 3000, nburnin = 1000, nchains = 4, inits =
  ↪ inits_fun,
                setSeed = TRUE, progressBar = FALSE, samplesAsCodaMCMC = TRUE)

summarise_draws(as_draws(fit_af), "mean", "sd", "rhat", "ess_bulk", "ess_tail")
```

```
# A tibble: 2 x 6
  variable mean      sd rhat ess_bulk ess_tail
  <chr>    <dbl>   <dbl> <dbl>   <dbl>   <dbl>
1 b0      -7.73   1.50   1.00   7312.   5840.
2 b1       0.157 0.0322  1.00   7392.   6110.
```

The joint slice sampler adapts to the correlation and mixes well without reparameterisation. Centring is still preferable: it costs nothing, works for any sampler and makes priors easier to specify.

(d)

```
or10 <- exp(as.matrix(fit_c)[, "b1"])
rbind(equal_tailed = quantile(or10, c(0.5, 0.025, 0.975)),
      HPD = c(median(or10), hdi(or10)))
```

```
              50%      2.5%      97.5%
equal_tailed 4.178154 2.491318 7.851108
HPD          4.178154 2.253982 7.258525
```

```
g <- glm(seropos ~ age10_c, family = binomial)
exp(c(coef(g)[2], confint.default(g)[2, ]))
```

```
age10_c      2.5 %    97.5 %
4.642516 2.474603 8.709658
```

The Bayesian estimate is somewhat smaller than the `glm()` estimate: the $\mathcal{N}(0,1)$ prior on the log-OR per 10 years gently shrinks towards $\text{OR} = 1$, which matters here because the effect is large and $n = 100$ is modest. (Is that prior appropriate for age and seropositivity? This is exactly the kind of choice to justify and check with a sensitivity analysis.) The OR posterior is right-skewed, so the HPD interval is shifted slightly to the left of the equal-tailed interval. The equal-tailed interval is transformation-invariant (it equals the exponential of the interval for β_1), which is why it is usually reported for odds ratios.